

Experiment 'p_14_7_bubble_sort_array' Results

June 5, 2026

Experiment outcome: FAILURE_RESOLVE

Bad responses: 1

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 3

Soft fails (resolution): 0

Verification attempts: 0

Problem Specification

Problem name: p_14_7_bubble_sort_array

Natural language statement: null

Method signature: p_14_4_bubble_sort_array(inputs: array<int>) returns (sorted: array<int>)

Ensures

- forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
- multiset(inputs[..]) == multiset(sorted[..])

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_14_4_bubble_sort_array(inputs: array<int>) returns (sorted: array<int>)`

The method should respect the following contract:

`ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j], ensures`
`↪ multiset(inputs[..]) == multiset(sorted[..])`

Produce and show only the Dafny body of this method, including the curly braces that
`↪` surround it. Do not show the signature nor contract. You must not use 'assume' nor
`↪` 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQFvvDUEu0udUCUyY51CmFZThqBB

Model: gpt-5.5-2026-04-23

Created at: 1780671371

Finish reason: stop

Usage: 310 tokens in, and 8355 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n](k => inputs[k]);

  assert sorted.Length == n;
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := n;
  while i > 0
    invariant 0 <= i <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases i
  {
    var j := 1;
    while j < i
```

```

invariant 1 <= j <= i
invariant sorted.Length == n
invariant multiset(sorted[..]) == multiset(inputs[..])
invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
invariant forall k :: 0 <= k < j ==> sorted[k] <= sorted[j - 1]
decreases i - j
{
  ghost var before := sorted[..];
  assert before == sorted[..];
  assert before.Length == n;
  assert multiset(before) == multiset(inputs[..]);
  assert forall k :: 0 <= k < j ==> before[k] <= before[j - 1];
  assert forall p, q :: i <= p < q < n ==> before[p] <= before[q];
  assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];

  if before[j] < before[j - 1] {
    var t := sorted[j - 1];
    sorted[j - 1] := sorted[j];
    sorted[j] := t;

    assert sorted[j - 1] == before[j];
    assert sorted[j] == before[j - 1];

    ghost var swapped := before[..j - 1] + [before[j]] + [before[j - 1]] + before[j +
      ↪ 1..];
    assert swapped.Length == before.Length;
    assert forall k | 0 <= k < sorted.Length
      ensures sorted[k] == swapped[k]
    {
      if k < j - 1 {
        assert sorted[k] == before[k];
        assert swapped[k] == before[k];
      } else if k == j - 1 {
        assert sorted[k] == before[j];
        assert swapped[k] == before[j];
      } else if k == j {
        assert sorted[k] == before[j - 1];
        assert swapped[k] == before[j - 1];
      } else {
        assert j + 1 <= k < sorted.Length;
        assert sorted[k] == before[k];
        assert swapped[k] == before[k];
      }
    }
  }
  assert sorted[..].Length == swapped.Length;
  assert forall k :: 0 <= k < sorted[..].Length ==> sorted[..][k] == swapped[k];
  assert sorted[..] == swapped;

  calc {
    multiset(sorted[..]);
  }
}

```

```

==
    multiset(swapped);
==
    multiset(before[..j - 1] + [before[j]] + [before[j - 1]] + before[j + 1..]);
==
    multiset(before[..j - 1]) + multiset([before[j]]) + multiset([before[j - 1]]) +
        ↪ multiset(before[j + 1..]);
==
    multiset(before[..j - 1]) + multiset([before[j - 1]]) + multiset([before[j]]) +
        ↪ multiset(before[j + 1..]);
==
    multiset(before[..j - 1] + [before[j - 1]] + [before[j]] + before[j + 1..]);
== {
    assert before == before[..j - 1] + [before[j - 1]] + [before[j]] + before[j +
        ↪ 1..];
}
    multiset(before);
}
} else {
    assert sorted[..] == before;
    assert multiset(sorted[..]) == multiset(before);
}

assert multiset(sorted[..]) == multiset(inputs[..]);

assert forall p, q | i <= p < q < n
    ensures sorted[p] <= sorted[q]
{
    assert sorted[p] == before[p];
    assert sorted[q] == before[q];
    assert before[p] <= before[q];
}

assert forall p, q | 0 <= p < i <= q < n
    ensures sorted[p] <= sorted[q]
{
    if before[j] < before[j - 1] {
        if p == j - 1 {
            assert sorted[p] == before[j];
            assert before[j] <= before[q];
        } else if p == j {
            assert sorted[p] == before[j - 1];
            assert before[j - 1] <= before[q];
        } else {
            assert sorted[p] == before[p];
            assert before[p] <= before[q];
        }
    } else {
        assert sorted[p] == before[p];
        assert before[p] <= before[q];
    }
}

```

```

    assert sorted[q] == before[q];
}

assert forall k | 0 <= k < j + 1
  ensures sorted[k] <= sorted[j]
{
  if before[j] < before[j - 1] {
    assert sorted[j] == before[j - 1];
    if k == j {
    } else if k == j - 1 {
      assert sorted[k] == before[j];
      assert before[j] < before[j - 1];
    } else {
      assert 0 <= k < j;
      assert sorted[k] == before[k];
      assert before[k] <= before[j - 1];
    }
  } else {
    assert before[j - 1] <= before[j];
    assert sorted[j] == before[j];
    if k == j {
    } else {
      assert 0 <= k < j;
      assert sorted[k] == before[k];
      assert before[k] <= before[j - 1];
      assert before[k] <= before[j];
    }
  }
}

j := j + 1;
}

assert j == i;
assert forall k :: 0 <= k < i ==> sorted[k] <= sorted[i - 1];

assert forall p, q | i - 1 <= p < q < n
  ensures sorted[p] <= sorted[q]
{
  if p == i - 1 {
    if q < i {
      assert q == i - 1;
    } else {
      assert i <= q < n;
      assert 0 <= i - 1 < i <= q < n;
      assert sorted[i - 1] <= sorted[q];
    }
  } else {
    assert i <= p < q < n;
    assert sorted[p] <= sorted[q];
  }
}

```

```

    }

    assert forall p, q | 0 <= p < i - 1 <= q < n
      ensures sorted[p] <= sorted[q]
    {
      if q == i - 1 {
        assert 0 <= p < i;
        assert sorted[p] <= sorted[i - 1];
      } else {
        assert i <= q < n;
        assert 0 <= p < i <= q < n;
        assert sorted[p] <= sorted[q];
      }
    }

    i := i - 1;
  }

  assert i == 0;
  assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
  assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_14_7_bubble_sort_array/program.dfy(89,6): Error: invalid UnaryExpression
|
89 | } else {
|   ^

```

```

problem_data/p_14_7_bubble_sort_array/program.dfy(88,8): Error: expected either 'by' or a
  ↳ semicolon following the assert expression
|
88 | }
|   ^

```

```

problem_data/p_14_7_bubble_sort_array/program.dfy(152,8): Error: invalid AssertStmt
|
152 | j := j + 1;
|   ^

```

3 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQlitSQ3LH9guG4fzL4tOuNArKBn

Model: gpt-5.5-2026-04-23

Created at: 1780671544

Finish reason: stop

Usage: 2504 tokens in, and 3363 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n](k => inputs[k]);

  assert sorted.Length == n;
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := n;
  while i > 0
    invariant 0 <= i <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases i
  {
    var j := 1;
    while j < i
      invariant 1 <= j <= i
      invariant sorted.Length == n
      invariant multiset(sorted[..]) == multiset(inputs[..])
      invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
      invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
      invariant forall k :: 0 <= k < j ==> sorted[k] <= sorted[j - 1]
      decreases i - j
    {
      ghost var before := sorted[..];
      assert before == sorted[..];
      assert before.Length == n;
      assert multiset(before) == multiset(inputs[..]);
      assert forall k :: 0 <= k < j ==> before[k] <= before[j - 1];
      assert forall p, q :: i <= p < q < n ==> before[p] <= before[q];
      assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];

      if before[j] < before[j - 1] {
        var t := sorted[j - 1];
        sorted[j - 1] := sorted[j];
        sorted[j] := t;

        assert sorted[j - 1] == before[j];
        assert sorted[j] == before[j - 1];
      }
    }
  }
}
```

```

ghost var swapped := before[..j - 1] + [before[j]] + [before[j - 1]] + before[j +
  ↪ 1..];
assert swapped.Length == before.Length;
assert forall k | 0 <= k < sorted.Length :: sorted[k] == swapped[k] by {
  if k < j - 1 {
    assert sorted[k] == before[k];
    assert swapped[k] == before[k];
  } else if k == j - 1 {
    assert sorted[k] == before[j];
    assert swapped[k] == before[j];
  } else if k == j {
    assert sorted[k] == before[j - 1];
    assert swapped[k] == before[j - 1];
  } else {
    assert j + 1 <= k < sorted.Length;
    assert sorted[k] == before[k];
    assert swapped[k] == before[k];
  }
}
assert sorted[..].Length == swapped.Length;
assert forall k :: 0 <= k < sorted[..].Length ==> sorted[..][k] == swapped[k];
assert sorted[..] == swapped;

calc {
  multiset(sorted[..]);
==
  multiset(swapped);
==
  multiset(before[..j - 1] + [before[j]] + [before[j - 1]] + before[j + 1..]);
==
  multiset(before[..j - 1]) + multiset([before[j]]) + multiset([before[j - 1]]) +
    ↪ multiset(before[j + 1..]);
==
  multiset(before[..j - 1]) + multiset([before[j - 1]]) + multiset([before[j]]) +
    ↪ multiset(before[j + 1..]);
==
  multiset(before[..j - 1] + [before[j - 1]] + [before[j]] + before[j + 1..]);
== {
  assert before == before[..j - 1] + [before[j - 1]] + [before[j]] + before[j +
    ↪ 1..];
}
  multiset(before);
}
} else {
  assert sorted[..] == before;
  assert multiset(sorted[..]) == multiset(before);
}

assert multiset(sorted[..]) == multiset(inputs[..]);

```



```

assert forall p, q | i <= p < q < n :: sorted[p] <= sorted[q] by {
  assert sorted[p] == before[p];
  assert sorted[q] == before[q];
  assert before[p] <= before[q];
}

assert forall p, q | 0 <= p < i <= q < n :: sorted[p] <= sorted[q] by {
  if before[j] < before[j - 1] {
    if p == j - 1 {
      assert sorted[p] == before[j];
      assert before[j] <= before[q];
    } else if p == j {
      assert sorted[p] == before[j - 1];
      assert before[j - 1] <= before[q];
    } else {
      assert sorted[p] == before[p];
      assert before[p] <= before[q];
    }
  } else {
    assert sorted[p] == before[p];
    assert before[p] <= before[q];
  }
  assert sorted[q] == before[q];
}

assert forall k | 0 <= k < j + 1 :: sorted[k] <= sorted[j] by {
  if before[j] < before[j - 1] {
    assert sorted[j] == before[j - 1];
    if k == j {
    } else if k == j - 1 {
      assert sorted[k] == before[j];
      assert before[j] < before[j - 1];
    } else {
      assert 0 <= k < j;
      assert sorted[k] == before[k];
      assert before[k] <= before[j - 1];
    }
  } else {
    assert before[j - 1] <= before[j];
    assert sorted[j] == before[j];
    if k == j {
    } else {
      assert 0 <= k < j;
      assert sorted[k] == before[k];
      assert before[k] <= before[j - 1];
      assert before[k] <= before[j];
    }
  }
}

j := j + 1;

```

```

}

assert j == i;
assert forall k :: 0 <= k < i ==> sorted[k] <= sorted[i - 1];

assert forall p, q | i - 1 <= p < q < n :: sorted[p] <= sorted[q] by {
  if p == i - 1 {
    if q < i {
      assert q == i - 1;
    } else {
      assert i <= q < n;
      assert 0 <= i - 1 < i <= q < n;
      assert sorted[i - 1] <= sorted[q];
    }
  } else {
    assert i <= p < q < n;
    assert sorted[p] <= sorted[q];
  }
}

assert forall p, q | 0 <= p < i - 1 <= q < n :: sorted[p] <= sorted[q] by {
  if q == i - 1 {
    assert 0 <= p < i;
    assert sorted[p] <= sorted[i - 1];
  } else {
    assert i <= q < n;
    assert 0 <= p < i <= q < n;
    assert sorted[p] <= sorted[q];
  }
}

i := i - 1;
}

assert i == 0;
assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_14_7_bubble_sort_array/program.dfy(34,20): Error: type seq<int> does not
  ↪ have a member Length
|
34 | assert before.Length == n;
|   ~~~~~~

problem_data/p_14_7_bubble_sort_array/program.dfy(49,23): Error: type seq<int> does not
  ↪ have a member Length

```

```

|
49 | assert swapped.Length == before.Length;
| ~~~~~

problem_data/p_14_7_bubble_sort_array/program.dfy(49,40): Error: type seq<int> does not
  ↪ have a member Length
|
49 | assert swapped.Length == before.Length;
| ~~~~~

problem_data/p_14_7_bubble_sort_array/program.dfy(51,13): Error: unresolved identifier: k
|
51 | if k < j - 1 {
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(52,26): Error: unresolved identifier: k
|
52 | assert sorted[k] == before[k];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(52,39): Error: unresolved identifier: k
|
52 | assert sorted[k] == before[k];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(53,27): Error: unresolved identifier: k
|
53 | assert swapped[k] == before[k];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(53,40): Error: unresolved identifier: k
|
53 | assert swapped[k] == before[k];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(54,20): Error: unresolved identifier: k
|
54 | } else if k == j - 1 {
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(55,26): Error: unresolved identifier: k
|
55 | assert sorted[k] == before[j];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(56,27): Error: unresolved identifier: k
|
56 | assert swapped[k] == before[j];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(57,20): Error: unresolved identifier: k

```

```

|
57 | } else if k == j {
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(58,26): Error: unresolved identifier: k
|
58 | assert sorted[k] == before[j - 1];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(59,27): Error: unresolved identifier: k
|
59 | assert swapped[k] == before[j - 1];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(61,28): Error: unresolved identifier: k
|
61 | assert j + 1 <= k < sorted.Length;
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(62,26): Error: unresolved identifier: k
|
62 | assert sorted[k] == before[k];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(62,39): Error: unresolved identifier: k
|
62 | assert sorted[k] == before[k];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(63,27): Error: unresolved identifier: k
|
63 | assert swapped[k] == before[k];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(63,40): Error: unresolved identifier: k
|
63 | assert swapped[k] == before[k];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(66,26): Error: type seq<?> does not
    ↪ have a member Length
|
66 | assert sorted[..].Length == swapped.Length;
| ~~~~~

problem_data/p_14_7_bubble_sort_array/program.dfy(66,44): Error: type seq<int> does not
    ↪ have a member Length
|
66 | assert sorted[..].Length == swapped.Length;
| ~~~~~

```

```

problem_data/p_14_7_bubble_sort_array/program.dfy(67,47): Error: type seq<?> does not
  ↪ have a member Length
  |
67 | assert forall k :: 0 <= k < sorted[..].Length ==> sorted[..][k] == swapped[k];
  | ~~~~~~

problem_data/p_14_7_bubble_sort_array/program.dfy(95,22): Error: unresolved identifier: p
  |
95 | assert sorted[p] == before[p];
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(95,35): Error: unresolved identifier: p
  |
95 | assert sorted[p] == before[p];
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(96,22): Error: unresolved identifier: q
  |
96 | assert sorted[q] == before[q];
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(96,35): Error: unresolved identifier: q
  |
96 | assert sorted[q] == before[q];
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(97,22): Error: unresolved identifier: p
  |
97 | assert before[p] <= before[q];
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(97,35): Error: unresolved identifier: q
  |
97 | assert before[p] <= before[q];
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(102,13): Error: unresolved identifier:
  ↪ p
  |
102 | if p == j - 1 {
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(103,26): Error: unresolved identifier:
  ↪ p
  |
103 | assert sorted[p] == before[j];
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(104,39): Error: unresolved identifier:
  ↪ q
  |

```

```

104 | assert before[j] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(105,20): Error: unresolved identifier:
    ↪ p
    |
105 | } else if p == j {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(106,26): Error: unresolved identifier:
    ↪ p
    |
106 | assert sorted[p] == before[j - 1];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(107,43): Error: unresolved identifier:
    ↪ q
    |
107 | assert before[j - 1] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(109,26): Error: unresolved identifier:
    ↪ p
    |
109 | assert sorted[p] == before[p];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(109,39): Error: unresolved identifier:
    ↪ p
    |
109 | assert sorted[p] == before[p];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(110,26): Error: unresolved identifier:
    ↪ p
    |
110 | assert before[p] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(110,39): Error: unresolved identifier:
    ↪ q
    |
110 | assert before[p] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(113,24): Error: unresolved identifier:
    ↪ p
    |
113 | assert sorted[p] == before[p];
    | ^

```

```

problem_data/p_14_7_bubble_sort_array/program.dfy(113,37): Error: unresolved identifier:
    ↪ p
    |
113 | assert sorted[p] == before[p];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(114,24): Error: unresolved identifier:
    ↪ p
    |
114 | assert before[p] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(114,37): Error: unresolved identifier:
    ↪ q
    |
114 | assert before[p] <= before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(116,22): Error: unresolved identifier:
    ↪ q
    |
116 | assert sorted[q] == before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(116,35): Error: unresolved identifier:
    ↪ q
    |
116 | assert sorted[q] == before[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(122,13): Error: unresolved identifier:
    ↪ k
    |
122 | if k == j {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(123,20): Error: unresolved identifier:
    ↪ k
    |
123 | } else if k == j - 1 {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(124,26): Error: unresolved identifier:
    ↪ k
    |
124 | assert sorted[k] == before[j];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(127,24): Error: unresolved identifier:
    ↪ k
    |

```

```

127 | assert 0 <= k < j;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(128,26): Error: unresolved identifier:
    ↪ k
    |
128 | assert sorted[k] == before[k];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(128,39): Error: unresolved identifier:
    ↪ k
    |
128 | assert sorted[k] == before[k];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(129,26): Error: unresolved identifier:
    ↪ k
    |
129 | assert before[k] <= before[j - 1];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(134,13): Error: unresolved identifier:
    ↪ k
    |
134 | if k == j {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(136,24): Error: unresolved identifier:
    ↪ k
    |
136 | assert 0 <= k < j;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(137,26): Error: unresolved identifier:
    ↪ k
    |
137 | assert sorted[k] == before[k];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(137,39): Error: unresolved identifier:
    ↪ k
    |
137 | assert sorted[k] == before[k];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(138,26): Error: unresolved identifier:
    ↪ k
    |
138 | assert before[k] <= before[j - 1];
    | ^

```



```

problem_data/p_14_7_bubble_sort_array/program.dfy(139,26): Error: unresolved identifier:
  ↪ k
  |
139 | assert before[k] <= before[j];
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(151,9): Error: unresolved identifier: p
  |
151 | if p == i - 1 {
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(152,11): Error: unresolved identifier:
  ↪ q
  |
152 | if q < i {
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(153,17): Error: unresolved identifier:
  ↪ q
  |
153 | assert q == i - 1;
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(155,22): Error: unresolved identifier:
  ↪ q
  |
155 | assert i <= q < n;
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(156,35): Error: unresolved identifier:
  ↪ q
  |
156 | assert 0 <= i - 1 < i <= q < n;
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(157,41): Error: unresolved identifier:
  ↪ q
  |
157 | assert sorted[i - 1] <= sorted[q];
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(160,20): Error: unresolved identifier:
  ↪ p
  |
160 | assert i <= p < q < n;
  | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(160,24): Error: unresolved identifier:
  ↪ q
  |
160 | assert i <= p < q < n;

```

```

| ^
problem_data/p_14_7_bubble_sort_array/program.dfy(161,22): Error: unresolved identifier:
  ↪ p
|
161 | assert sorted[p] <= sorted[q];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(161,35): Error: unresolved identifier:
  ↪ q
|
161 | assert sorted[p] <= sorted[q];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(166,9): Error: unresolved identifier: q
|
166 | if q == i - 1 {
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(167,20): Error: unresolved identifier:
  ↪ p
|
167 | assert 0 <= p < i;
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(168,22): Error: unresolved identifier:
  ↪ p
|
168 | assert sorted[p] <= sorted[i - 1];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(170,20): Error: unresolved identifier:
  ↪ q
|
170 | assert i <= q < n;
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(171,20): Error: unresolved identifier:
  ↪ p
|
171 | assert 0 <= p < i <= q < n;
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(171,29): Error: unresolved identifier:
  ↪ q
|
171 | assert 0 <= p < i <= q < n;
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(172,22): Error: unresolved identifier:
  ↪ p

```

```

|
172 | assert sorted[p] <= sorted[q];
| ^

problem_data/p_14_7_bubble_sort_array/program.dfy(172,35): Error: unresolved identifier:
  ↪ q
|
172 | assert sorted[p] <= sorted[q];
| ^

75 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
  ↪ Dafny body, including the curly braces that surround it. Do not show the signature
  ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQJq6YlgecxuSUFbqZK1e3Aa3YOA

Model: gpt-5.5-2026-04-23

Created at: 1780671614

Finish reason: stop

Usage: 11139 tokens in, and 3470 tokens out

```

{
  var n := inputs.Length;
  sorted := new int[n](k => inputs[k]);

  assert sorted.Length == n;
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := n;
  while i > 0
    invariant 0 <= i <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases i
  {
    var j := 1;
    while j < i
      invariant 1 <= j <= i
      invariant sorted.Length == n
      invariant multiset(sorted[..]) == multiset(inputs[..])
      invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
      invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
      invariant forall k :: 0 <= k < j ==> sorted[k] <= sorted[j - 1]

```

```

decreases i - j
{
  ghost var before := sorted[..];
  assert before == sorted[..];
  assert |before| == n;
  assert multiset(before) == multiset(inputs[..]);
  assert forall k :: 0 <= k < j ==> before[k] <= before[j - 1];
  assert forall p, q :: i <= p < q < n ==> before[p] <= before[q];
  assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];

  if before[j] < before[j - 1] {
    var t := sorted[j - 1];
    sorted[j - 1] := sorted[j];
    sorted[j] := t;

    assert sorted[j - 1] == before[j];
    assert sorted[j] == before[j - 1];

    ghost var swapped := before[..j - 1] + [before[j]] + [before[j - 1]] + before[j +
      ↪ 1..];
    assert |swapped| == |before|;
    forall k | 0 <= k < sorted.Length
      ensures sorted[k] == swapped[k]
    {
      if k < j - 1 {
        assert sorted[k] == before[k];
        assert swapped[k] == before[k];
      } else if k == j - 1 {
        assert sorted[k] == before[j];
        assert swapped[k] == before[j];
      } else if k == j {
        assert sorted[k] == before[j - 1];
        assert swapped[k] == before[j - 1];
      } else {
        assert j + 1 <= k < sorted.Length;
        assert sorted[k] == before[k];
        assert swapped[k] == before[k];
      }
    }
  }
  assert |sorted[..]| == |swapped|;
  assert forall k :: 0 <= k < |sorted[..]| ==> sorted[..][k] == swapped[k];
  assert sorted[..] == swapped;

  calc {
    multiset(sorted[..]);
  ==
    multiset(swapped);
  ==
    multiset(before[..j - 1] + [before[j]] + [before[j - 1]] + before[j + 1..]);
  ==
    multiset(before[..j - 1]) + multiset([before[j]]) + multiset([before[j - 1]]) +

```

```

    ↪ multiset(before[j + 1..]);
==
    multiset(before[..j - 1]) + multiset([before[j - 1]]) + multiset([before[j]]) +
    ↪ multiset(before[j + 1..]);
==
    multiset(before[..j - 1] + [before[j - 1]] + [before[j]] + before[j + 1..]);
== {
    assert before == before[..j - 1] + [before[j - 1]] + [before[j]] + before[j +
    ↪ 1..];
}
    multiset(before);
}
} else {
    assert sorted[..] == before;
    assert multiset(sorted[..]) == multiset(before);
}

assert multiset(sorted[..]) == multiset(inputs[..]);

forall p, q | i <= p < q < n
    ensures sorted[p] <= sorted[q]
{
    assert sorted[p] == before[p];
    assert sorted[q] == before[q];
    assert before[p] <= before[q];
}

forall p, q | 0 <= p < i <= q < n
    ensures sorted[p] <= sorted[q]
{
    if before[j] < before[j - 1] {
        if p == j - 1 {
            assert sorted[p] == before[j];
            assert before[j] <= before[q];
        } else if p == j {
            assert sorted[p] == before[j - 1];
            assert before[j - 1] <= before[q];
        } else {
            assert sorted[p] == before[p];
            assert before[p] <= before[q];
        }
    } else {
        assert sorted[p] == before[p];
        assert before[p] <= before[q];
    }
    assert sorted[q] == before[q];
}

forall k | 0 <= k < j + 1
    ensures sorted[k] <= sorted[j]
{

```

```

    if before[j] < before[j - 1] {
      assert sorted[j] == before[j - 1];
      if k == j {
      } else if k == j - 1 {
        assert sorted[k] == before[j];
        assert before[j] < before[j - 1];
      } else {
        assert 0 <= k < j;
        assert sorted[k] == before[k];
        assert before[k] <= before[j - 1];
      }
    } else {
      assert before[j - 1] <= before[j];
      assert sorted[j] == before[j];
      if k == j {
      } else {
        assert 0 <= k < j;
        assert sorted[k] == before[k];
        assert before[k] <= before[j - 1];
        assert before[k] <= before[j];
      }
    }
  }
}

j := j + 1;
}

assert j == i;
assert forall k :: 0 <= k < i ==> sorted[k] <= sorted[i - 1];

forall p, q | i - 1 <= p < q < n
  ensures sorted[p] <= sorted[q]
{
  if p == i - 1 {
    if q < i {
      assert q == i - 1;
    } else {
      assert i <= q < n;
      assert 0 <= i - 1 < i <= q < n;
      assert sorted[i - 1] <= sorted[q];
    }
  } else {
    assert i <= p < q < n;
    assert sorted[p] <= sorted[q];
  }
}

forall p, q | 0 <= p < i - 1 <= q < n
  ensures sorted[p] <= sorted[q]
{
  if q == i - 1 {

```

```

    assert 0 <= p < i;
    assert sorted[p] <= sorted[i - 1];
  } else {
    assert i <= q < n;
    assert 0 <= p < i <= q < n;
    assert sorted[p] <= sorted[q];
  }
}

i := i - 1;
}

assert i == 0;
assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_14_7_bubble_sort_array/program.dfy(42,22): Error: assignment to array
  ↳ element is not allowed in this context, because the statement is in a ghost
  ↳ context; e.g., it may be guarded by a specification-only expression

```

```

42 | sorted[j - 1] := sorted[j];
   | ^^

```

```

problem_data/p_14_7_bubble_sort_array/program.dfy(43,18): Error: assignment to array
  ↳ element is not allowed in this context, because the statement is in a ghost
  ↳ context; e.g., it may be guarded by a specification-only expression

```

```

43 | sorted[j] := t;
   | ^^

```

2 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQL3ZKwdCwOk3GPmNzN8LwdddU0f

Model: gpt-5.5-2026-04-23

Created at: 1780671689

Finish reason: length

Usage: 10037 tokens in, and 859 tokens out

The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
method p_14_4_bubble_sort_array(inputs: array<int>) returns (sorted: array<int>)
  ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
  ensures multiset(inputs[..]) == multiset(sorted[..])
{
  var n := inputs.Length;
  sorted := new int[n](k => inputs[k]);

  assert sorted.Length == n;
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := n;
  while i > 0
    invariant 0 <= i <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases i
  {
    var j := 1;
    while j < i
      invariant 1 <= j <= i
      invariant sorted.Length == n
      invariant multiset(sorted[..]) == multiset(inputs[..])
      invariant forall p, q :: i <= p < q < n ==> sorted[p] <= sorted[q]
      invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
      invariant forall k :: 0 <= k < j ==> sorted[k] <= sorted[j - 1]
      decreases i - j
    {
      ghost var before := sorted[..];
      assert before == sorted[..];
      assert |before| == n;
      assert multiset(before) == multiset(inputs[..]);
      assert forall k :: 0 <= k < j ==> before[k] <= before[j - 1];
      assert forall p, q :: i <= p < q < n ==> before[p] <= before[q];
      assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];

      if before[j] < before[j - 1] {
        var t := sorted[j - 1];
        sorted[j - 1] := sorted[j];
        sorted[j] := t;

        assert sorted[j - 1] == before[j];
        assert sorted[j] == before[j - 1];
```



```

ghost var swapped := before[..j - 1] + [before[j]] + [before[j - 1]] + before[j +
  ↪ 1..];
assert |swapped| == |before|;
forall k | 0 <= k < sorted.Length
  ensures sorted[k] == swapped[k]
{
  if k < j - 1 {
    assert sorted[k] == before[k];
    assert swapped[k] == before[k];
  } else if k == j - 1 {
    assert sorted[k] == before[j];
    assert swapped[k] == before[j];
  } else if k == j {
    assert sorted[k] == before[j - 1];
    assert swapped[k] == before[j - 1];
  } else {
    assert j + 1 <= k < sorted.Length;
    assert sorted[k] == before[k];
    assert swapped[k] == before[k];
  }
}
assert |sorted[..]| == |swapped|;
assert forall k :: 0 <= k < |sorted[..]| ==> sorted[..][k] == swapped[k];
assert sorted[..] == swapped;

calc {
  multiset(sorted[..]);
==
  multiset(swapped);
==
  multiset(before[..j - 1] + [before[j]] + [before[j - 1]] + before[j + 1..]);
==
  multiset(before[..j - 1]) + multiset([before[j]]) + multiset([before[j - 1]]) +
    ↪ multiset(before[j + 1..]);
==
  multiset(before[..j - 1]) + multiset([before[j - 1]]) + multiset([before[j]]) +
    ↪ multiset(before[j + 1..]);
==
  multiset(before[..j - 1] + [before[j - 1]] + [before[j]] + before[j + 1..]);
== {
  assert before == before[..j - 1] + [before[j - 1]] + [before[j]] + before[j +
    ↪ 1..];
}
  multiset(before);
}
} else {
  assert sorted[..] == before;
  assert multiset(sorted[..]) == multiset(before);
}

assert multiset(sorted[..]) == multiset(inputs[..]);

```

```

forall p, q | i <= p < q < n
  ensures sorted[p] <= sorted[q]
{
  assert sorted[p] == before[p];
  assert sorted[q] == before[q];
  assert before[p] <= before[q];
}

forall p, q | 0 <= p < i <= q < n
  ensures sorted[p] <= sorted[q]
{
  if before[j] < before[j - 1] {
    if p == j - 1 {
      assert sorted[p] == before[j];
      assert before[j] <= before[q];
    } else if p == j {
      assert sorted[p] == before[j - 1];
      assert before[j - 1] <= before[q];
    } else {
      assert sorted[p] == before[p];
      assert before[p] <= before[q];
    }
  } else {
    assert sorted[p] == before[p];
    assert before[p] <= before[q];
  }
  assert sorted[q] == before[q];
}

forall k | 0 <= k < j + 1
  ensures sorted[k] <= sorted[j]
{
  if before[j] < before[j - 1] {
    assert sorted[j] == before[j - 1];
    if k == j {
    } else if k == j - 1 {
      assert sorted[k] == before[j];
      assert before[j] < before[j - 1];
    } else {
      assert 0 <= k < j;
      assert sorted[k] == before[k];
      assert before[k] <= before[j - 1];
    }
  } else {
    assert before[j - 1] <= before[j];
    assert sorted[j] == before[j];
    if k == j {
    } else {
      assert 0 <= k < j;
      assert sorted[k] == before[k];
    }
  }
}

```

```

        assert before[k] <= before[j - 1];
        assert before[k] <= before[j];
    }
}

j := j + 1;
}

assert j == i;
assert forall k :: 0 <= k < i ==> sorted[k] <= sorted[i - 1];

forall p, q | i - 1 <= p < q < n
    ensures sorted[p] <= sorted[q]
{
    if p == i - 1 {
        if q < i {
            assert q == i - 1;
        } else {
            assert i <= q < n;
            assert 0 <= i - 1 < i <= q < n;
            assert sorted[i - 1] <= sorted[q];
        }
    } else {
        assert i <= p < q < n;
        assert sorted[p] <= sorted[q];
    }
}

forall p, q | 0 <= p < i - 1 <= q < n
    ensures sorted[p] <= sorted[q]
{
    if q == i - 1 {
        assert 0 <= p < i;
        assert sorted[p] <= sorted[i - 1];
    } else {
        assert i <= q < n;
        assert 0 <= p < i <= q < n;
        assert sorted[p] <= sorted[q];
    }
}

i := i - 1;
}

assert i == 0;
assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Total Token Usage

Input tokens: 23990

Output tokens: 16047

Reasoning tokens: 10203

Sum of ‘total tokens’: 40037

Experiment Timings

Overall Experiment started at 1780671369432, ended at 1780671712254, lasting 342822ms (342.82 seconds)

Iteration #4 started at 1780671688694, ended at 1780671712253, lasting 23559ms (23.56 seconds)

Iteration #1 started at 1780671369446, ended at 1780671543250, lasting 173804ms (173.80 seconds)

Iteration #2 started at 1780671543250, ended at 1780671613783, lasting 70533ms (70.53 seconds)

Iteration #3 started at 1780671613783, ended at 1780671688694, lasting 74911ms (74.91 seconds)